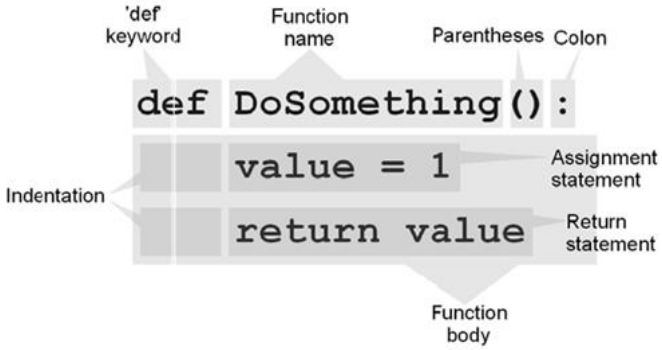
LAB#02

FUNCTIONS, LIST, TUPLE AND DICTIONARY

Familiarization with Python language using functions, list, tuple and dictionary.

I. Function

A function is a group of statements that is executed when it is called from some point of the program. The following is its format:



where:

def is a keyword to define a function.

Function name is the identifier by which it will be possible to call the function.

Parameters (as many a needed): each parameter consists of an identifier, like any regular variable and which acts within the functions as a regular local variable. They separated by commas.

Statements is the function body it is a block of statements justified by indentation.

Consider the following code and examine the repeated code with same functionality.

```
for i in range(30):
    print("*" , end='')
print()
print("Artificial Intelligence")
for i in range(30):
    print("*",end='')
print()
print("Lab 02")
for i in range(30):
    print("*", end='')
print()
print("Department of Computer Science")
for i in range(30):
    print("*",end='')
print()
```

a) Functions Without Return Type and No Argument:

```
def function_name ( ):
```

What we will do in this case for similar functionality code, create a function named drawLine() and call that function at repeated code locations.

```
def drawLine():
    for i in range(30):
        print("*" , end='')
    print()

drawLine()
print("Artificial Intelligence")
drawLine()
print("Lab02")
drawLine()
print("Department of Computer Science")
drawLine()
```

b) Functions without Return Type and with Argument:

```
def function_name(arg1,arg2,arg3):
```

drawLine function with an argument to draw line according to the size provided

```
def drawLine(n):
    for i in range(n):
        print("*" , end='')
    print()

drawLine(30)
print("Artificial Intelligence")
drawLine(10)
print("Lab02")
drawLine(10)
print("Department of Computer Science")
drawLine(30)
```

c) Functions with Return Type and no Argument:

```
def function_name ( ):
```

```
def getUniversityName():
    return "Sir Syed University"
     print("The  University Name is ", getUniversityName())
```

d) Functions with Return Type and Argument:

```
def function_name(arg1,arg2,arg3):
```

```
def findSum(b):
    total=0
    for i in range(len(b)):
        total+=b[i]
    return total

a = [2,3,5,8,4,9,7,6,7,8]
sum = findSum(a)
print ("The result is ", sum)
```

e) Pass by Reference (value of memory address location):

All parameters (arguments) in the Python language are passed by reference. It means if you change what a parameter refers to within a function, the change also reflects back in the calling function.

f) Functions with Default value Arguments:

```
def function_name(arg1 = value, arg2 = value, ....):
```

```
def sum(a=5,b=10,c=20):
    return a+b+c;

print("The sum without passing parameters",sum())
print("The sum with one parameter",sum(10))
print("The sum with two parameters",sum(10,20))
print("The sum with all parameters",sum(10,20,30))
```

g) Recursive function:

"A *recursive function* is a function that either directly or indirectly makes a call to itself.". It does this by making problem smaller (simpler) at each call.

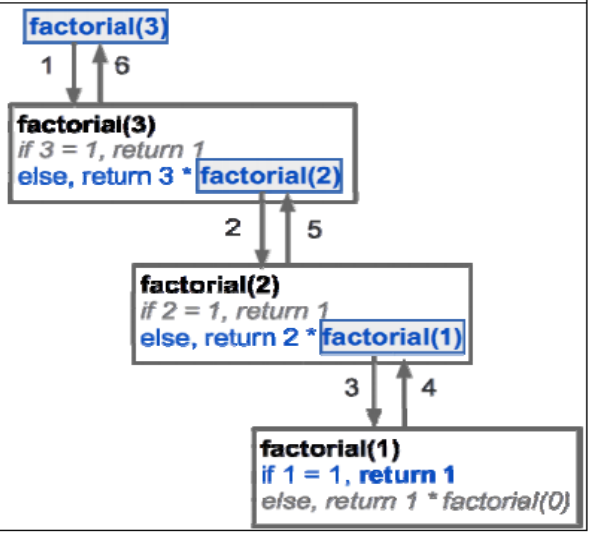
Recursive functions have two important components:

- **Base case(s)**, where the function directly computes an answer without calling itself. Usually, the base case deals with the simplest possible form of the problem you're trying to solve.
- **Recursive case(s)**, where the function calls itself as part of the computation.

s

Iterative Solution	Recursive Solution

`def factorial( n ):`
`  sum = 1`
`  for i in range(1,n+1):`
`  sum *= i;`
`  return sum;`
 	
`print("Factorial of 3 is " ,factorial(3))`
	`def factorial( n ):`
	`  if( n <= 1 ):`
	`  return 1`
	`  else:`
	`  return n * factorial( n-1 )`
	
	`print("Factorial of 3 is " ,factorial(6))`

`sum =1*1=1`
`sum =1*2=2`
`sum =2*3=6`
`return sum=6`
	

For example:

factorial(5) = 5 * factorial(4)

= 5 * (4 * factorial(3))

= 5 * (4 * (3 * factorial(2)))

= 5 * (4 * (3 * (2 * factorial(1))))

= 5 * (4 * (3 * (2 * (1 * factorial(0)))))

= 5 * (4 * (3 * (2 * (1 * 1))))

= 5 * 4 * 3 * 2 * 1 * 1 = 120

II. List

A *list* is a collection of items in a particular order. You can make a list that includes the letters of the alphabet, the digits from 0–9, or the names of all the people in your family. You can put anything you want into a list, and the items in your list don't have to be related in any particular way. Because a list usually contains more than one element, it's a good idea to make the name of your list plural, such as letters, digits, or names.
It can have any number of items and they may be of different types (integer, float, string etc.). In Python, square brackets ([]) indicate a list, and individual elements in the list are separated by commas.

```
# empty list
my_list = []
# list of integers
my_list = [1, 2, 3]
# list with mixed datatypes
my_list = [1, "Hello", 3.4]
```

Also, a list can even have another list as an item. This is called nested list.

```
# nested list
my_list = ["mouse", [8, 4, 6], ['a']]
```

Some simple examples of a list:

```
>>> my_list = ['p','r','o','b','e']
>>> print(my_list[0])
p
>>> print(my_list[2])
o
>>> n_list = ["Happy", [2,0,1,5]]
>>> print(n_list[0][1])
a
>>> a = [["a","b","c",],["d","e"],["f","g","h"]]
>>> print(a[0][1])
b
>>> print(a[2][1])
g

>>> my_list = ['p','r','o','b','e']
>>> print(my_list[-5])
p
>>> print(my_list[2:5])
['o', 'b', 'e']
>>> print(my_list[:])
['p', 'r', 'o', 'b', 'e']
>>> print(my_list[5:])
[]
>>> my_list = ['p','r','o','g','r','a','m','i','z']
>>> print(my_list[5:])
['a', 'm', 'i', 'z']
```

a) Accessing Values in Lists:

To access values in lists, use the square brackets for slicing along with the index or indices to obtain value available at that index. Python considers the first item in a list to be at position 0, not position 1

Example:

```
>>> list1 = ['physics', 'chemistry', 1997, 2000]
>>> list2 = [1, 2, 3, 4, 5, 6, 7 ]
>>> print ("list1[0]: ", list1[0])
list1[0]:  physics
>>> print ("list2[1:5]: ", list2[1:5])
list2[1:5]:  [2, 3, 4, 5]
>>> print ("list1[-2]: ", list1[-2])
list1[-2]:  1997
>>>
```

b) Modifying Elements in a List:

The syntax for modifying an element is similar to the syntax for accessing an element in a list. To change an element, use the name of the list followed by the index of the element you want to change, and then provide the new value you want that item to have. For example, let's say we have a list of motorcycles, and the first item in the list is 'honda'. How would we change the value of this first item?

Example:

File Edit Format Run Options Windows Help

```
motorcycles = ['honda', 'yamaha', 'suzuki']
print(motorcycles)
motorcycles[0] = 'ducati'
print(motorcycles)
```

Output:

```
>>>
['honda', 'yamaha', 'suzuki']
['ducati', 'yamaha', 'suzuki']
>>>
```

c) Adding/Appending Elements to a List:

You might want to add a new element to a list for many reasons. For example, you might want to make new aliens appear in a game, add new data to visualization, or add new registered users to a website you've built. Python provides several ways to add new data to existing lists.

The simplest way to add a new element to a list is to *append* the item to the list. When you append an item to a list, the new element is added to the end of the list. Using the same list we had in the previous example, we'll add the new element 'ducati' to the end of the list:

Example:

```
File  Edit  Format  Run  Options  Windows  Help
motorcycles = ['honda', 'yamaha', 'suzuki']
print(motorcycles)
motorcycles.append('ducati')
print(motorcycles)
```

The append () method, adds 'ducati' to the end of the list without affecting any of the other elements in the list:

Output:

```
>>>
['honda', 'yamaha', 'suzuki']
['honda', 'yamaha', 'suzuki', 'ducati']
>>>
```

d) Removing Elements from a List:

Remove an item or a set of items from a list using different function like remove(),del() etc. For example using remove(),You can remove an item according to its position in the list or according to its value.

Example:

```
oo.py - C:/Python32/py/oo.py
File  Edit  Format  Run  Options  Windows  Help
aList = [123, 'xyz', 'zara', 'abc', 'xyz']
aList.remove('xyz')
print ("List : ", aList)
```

Output:

```
>>>
List :  [123, 'zara', 'abc', 'xyz']
>>>
```

- The remove operation on a list is given a value to remove. It searches the list to find an item with that value and deletes the first matching item it finds. It is an error if there is no matching item.

- The del statement can be used to delete an entire list. If you have a specific list item as your argument to del. It is even possible to delete a "slice" from a list.

e) Sorting a List Permanently with the sort() Method:

Python's sort() method makes it relatively easy to sort a list. Imagine we have a list of cars and want to change the order of the list to store them alphabetically. To keep the task simple, let's assume that all the values in the list are lowercase.

Example:

```
File  Edit  Format  Run  Options  Windows  Help
cars = ['bmw', 'audi', 'toyota', 'subaru']
cars.sort()
print(cars)
```

Output:

```
>>>
['audi', 'bmw', 'subaru', 'toyota']
>>>
```

f) Built-in List Functions & Methods:

Python includes the following list functions:

Sr.No.	Function with Description
1	**cmp(list1, list2)** Compares elements of both lists.
2	**len(list)** Gives the total length of the list.
3	**max(list)** Returns item from the list with max value.
4	**min(list)** Returns item from the list with min value.
5	**list(seq)** Converts a tuple into list.

Python includes following list methods

Sr.No.	Methods with Description
1	**list.append(obj)** Appends object obj to list
2	**list.count(obj)** Returns count of how many times obj occurs in list
3	**list.extend(seq)** Appends the contents of seq to list
4	**list.index(obj)** Returns the lowest index in list that obj appears

5	**list.insert(index, obj)** Inserts object obj into list at offset index
6	**list.pop(obj=list[-1])** Removes and returns last object or obj from list
7	**list.remove(obj)** Removes object obj from list
8	**list.reverse()** Reverses objects of list in place
9	**list.sort([func])** Sorts objects of list, use compare func if given

III. Tuples

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally you can put these comma-separated values between parentheses also. For example −

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5);
tup3 = "a", "b", "c", "d";
tup4 = ( );
```

To write a tuple containing a single value you have to include a comma, even though there is only one value –

```
tup1 = (50,);
```

Example:

```
File  Edit  Format  Run  Options  Windows  Help
dimensions = (200, 50)
print("Original dimensions:")
for dimension in dimensions:
 print(dimension)
dimensions = (400, 100)
print("\nModified dimensions:")
for dimension in dimensions:
 print(dimension)
```

The block at line 1, defines the original tuple and prints the initial dimensions. At line 5, we store a new tuple in the variable dimensions. We then print the new dimensions at line 6. Python doesn't raise any errors this time, because overwriting a variable is valid:

Output:

```
Original dimensions:
200
50

Modified dimensions:
400
100
>>>
```

IV. Dictionary

A dictionary in Python is a collection of key-value pairs. Each key is connected to a value, and you can use a key to access the value associated with that key. A key's value can be a number, a string, a list, or even another dictionary. Python dictionary is an unordered collection of items. While other compound data types have only value as an element, a dictionary has a key: value pair. Dictionaries can store an almost limitless amount of information.

Creating a dictionary is as simple as placing items inside curly braces {} separated by comma. An item has a key and the corresponding value expressed as a pair, key: value.

```
dict = {'color': 'green', 'points': 5}
```

a) Adding New Key-Value Pairs:

Dictionaries are dynamic structures, and you can add new key-value pairs to a dictionary at any time. For example, to add a new key-value pair, you would give the name of the dictionary followed by the new key in square brackets along with the new value.

Example:

```
dict = {'color': 'green', 'points': 5}
print(dict)
dict['x_position'] = 0
dict['y_position'] = 25
print(dict)
```

Output:

```
{'color': 'green', 'points': 5}
{'color': 'green', 'points': 5, 'x_position': 0, 'y_position': 25}
>>>
```

Example:

```
d ={}
while True:
    name = input('Enter name of Employee ')
    salary = int(input('Enter salary '))
    d[name]=salary
    ans = input('Wants to Enter more record y/n? ')
    if ans.lower()=='n':
        break
print(d)
```

b) Removing Key-value pair:

When you no longer need a piece of information that's stored in a dictionary, you can use the del statement to completely remove a key-value pair. All 'del' needs is the name of the dictionary and the key that you want to remove.

Example:

```
dict = {'color': 'green', 'points': 5}
print(dict)
del dict['points']
print(dict)
```

Output:

```
{'color': 'green', 'points': 5}
{'color': 'green'}
>>>
```

c) Looping Through a Dictionary:

A single Python dictionary can contain just a few key-value pairs or millions of pairs. Dictionaries can be used to store information in a variety of ways; therefore, several different ways exist to loop through them. You can loop through all of a dictionary's key-value pairs, through its keys, or through its values.

Example:

```
user = {
'username': 'ef01',
'first': 'enric',
'last': 'abc12',
}
for key, value in user.items():
 print("\nKey: "  +key)
 print("Value: "  +value)
```

- ✓ The method items () returns a list of dictionary's (key, value) tuple pairs. The syntax of items () method is: dictionary.items ().
- ✓ The key-value pairs are not returned in the order in which they were stored, even when looping through a dictionary. Python doesn't care about the order in which key-value pairs are stored; it tracks only the connections between individual keys and their values.

Output:

```
Key: username
Value: ef01

Key: last
Value: abc12

Key: first
Value: enric
>>>
```

Activity- 1:

Write a Python script that uses a dictionary to store information about a person you know. Store their first name, last name, age, and the city in which they live. You should have keys such as first_name, last_name, age, and city. Print each piece of information stored in your dictionary.

Activity- 2:

Write a function that accepts a dictionary as an argument. If the dictionary contains replicate values, return an empty dictionary, otherwise, return a new dictionary whose values are now the keys and whose keys are the values.

Activity- 3:

A buffet-style restaurant offers only five basic foods. Think of five simple foods, and store them in a tuple.

- Use a for loop to print each food the restaurant offers.
- Try to modify one of the items, and make sure that Python rejects the change.

Activity- 4:

If you could invite anyone to dinner. Make a list that includes at least three people you'd like to invite to dinner. Then use your list to print a message to each person, inviting them to dinner.

Activity- 5:

Changing Guest List: You just heard that one of your guests can't make the dinner, so you need to send out a new set of invitations. You'll have to think of someone else to invite.

- Modify your list, replacing the name of the guest who can't make it with the name of the new person you are inviting.
- Print a second set of invitation messages, one for each person who is still in your list.

Activity- 6:

Write a program to read 6 numbers and create a dictionary having keys EVEN and ODD. Dictionary's value should be stored in list. Your dictionary should be like: {'EVEN':[8,10,64], 'ODD':[1,5,9]}

Activity- 7:

Write a definition of a method count_now(places) to find and display those place names, in which there are more than 5 characters.

Activity- 8:

Write the following 2 functions:

```
def   ComputeOddSum(num):
def ComputeEvenSum(num):
```

- The function **`ComputeOddSum`** find the sum of all odd numbers less than num.
- The function **`ComputeEvenSum`** find the sum of all even numbers less than num.

Activity- 9:

Write a recursive function to get sum of all number from 1 up to give number. Example N = 5 Result must be sum (1+2+3+4+5) =15.